

CockroachDB Distributed SQL

L0 Essence: The essence of a distributed SQL database is to perform horizontal range partitioning and Multi-Raft consensus replication, utilizing Hybrid Logical Clocks (HLC) and MVCC to achieve distributed transactional consistency and snapshot isolation, rendering a shared-nothing cluster as a single-instance relational SQL database.

[M1.1] SQL Parsing and Execution: Two-Phase Distribution at the Gateway Node

The Gateway node parses SQL to construct an Abstract Syntax Tree (AST), performs semantic analysis, and rewrites it into a logical plan. The optimizer identifies target KV Range boundaries, queries the Meta index tree to resolve physical nodes, and splits the logical plan into multiple physical sub-plans sent to individual nodes for execution.

[M1.2] DistSQL Execution Engine: Streaming Pipelines and Flow Topology

The DistSQL engine converts distributed physical plans into execution Flows. A Flow consists of multiple local Processors (e.g., TableReader, Joiner) interconnected by network Routers. Tuples flow through streaming pipelines in batches using a Push/Pull model, eliminating bottlenecks and high bandwidth load at the Gateway node.

[M1.3] Distributed Scan Operator and Primary/Secondary Index KV Encoding

CockroachDB maps relational table rows to a flat KV namespace. The Primary Key is encoded as: `Prefix/TableID/IndexID/PrimaryKey` $\rightarrow$ `ColumnValues`. The Secondary Index is encoded as: `Prefix/TableID/IndexID/SecondaryKey/PrimaryKey` $\rightarrow$ `void`. The Scan operator performs rapid ranges queries based on prefix matching.

[M1.4] Optimizer: Cost-Based Path Selection (CBO) and Statistics Maintenance

The cost-based optimizer (CBO) evaluates physical plans using a Cascades framework. By tracking equi-depth histograms, multi-column correlation statistics, and physical latency profiles across nodes, the CBO calculates the cost of Join algorithms (Lookup Join, Merge Join, Hash Join) to select the execution topology with minimal network overhead.

[M2.1] Range Partitioning: Dynamic Splits and Merges

Table rows are sorted by primary key and divided into contiguous Ranges, capped at 64MB by default. When continuous writes exceed this threshold or create localized write hot-spots, nodes initiate local splits to partition data and register new child Ranges in the global 3-level Meta index tree. Conversely, empty or shrinking Ranges trigger merges.

[M2.2] Multi-Raft Consensus Architecture: Heartbeat Coalescing and Isolation

Each Range constitutes a separate Raft Group. With a single node hosting tens of thousands of replicas, standard heartbeats would trigger cluster-wide "heartbeat storms". The Multi-Raft engine aggregates heartbeats destined for the same physical host into single network packets (Coalesced Heartbeats), slashing network IO and CPU interrupts.

[M2.3] Leaseholder Mechanism: Read/Write Flow and Brain-Split Prevention

To avoid the consensus overhead of Raft majority reads, each Range elects a Leaseholder (typically co-located with the Raft Leader). All read/write requests must route to the Leaseholder. The Leaseholder serves linearizable reads locally, whereas write requests are proposed to the Raft group, bypassing brain-split hazards.

[M2.4] Replica Rebalancing and Raft Membership Change Control

The Allocator monitors node-level resource metrics. When capacity watermarks or CPU utilization diverge, the Allocator calculates optimal target nodes based on physical topology and executes replica relocation using Raft Joint Consensus, ensuring seamless read/write execution during migration.

[M3.1] Distributed 2PC: Atomicity and the Transaction Record

Distributed transactions use Two-Phase Commit (2PC). The coordinator creates a Transaction Record in the system Range with a PENDING state. Phase 1 writes data replicas across target ranges; Phase 2 atomically toggles the Record status to COMMITTED, cementing transaction safety even if coordinator processes fail before cleanup.

[M3.2] Write Intent Mechanism and Row-Level Concurrency Conflict Detection

Writes do not directly overwrite MVCC values; they write a Write Intent—a temporary record containing the proposed value alongside a pointer to the Transaction Record, acting as an active exclusive lock. Concurrent readers verify the status of the Transaction Record via the pointer to evaluate the visibility of the Intent.

[M3.3] Two-Phase Commit Optimization: One-Phase Commit (1PC)

If a transaction's mutated keys are localized within a single Range, the coordinator bypasses the distributed 2PC protocol. Transaction execution and commit operations are coalesced into a single Raft log entry inside the local Multi-Raft Group, shrinking transaction latency to a single local consensus round.

[M3.4] Transaction Cleanup: Intent Resolution and Async Garbage Collection

Once a Transaction Record transitions to COMMITTED or ABORTED, the coordinator triggers an asynchronous cleanup task (Intent Resolution) to convert Write Intents into committed physical versions (or purge them to release row locks), unblocking concurrent transactions immediately.

L1 Logical Framework

1. Data Partitioning & Consensus: Tables are automatically partitioned into 64MB

Ranges, each mapped to a Multi-Raft Group and mediated by a Leaseholder to intercept read/write traffic.

2. Transactions & Intent Locking: Distributed transactions are managed via Two-Phase Commit (2PC) updating a single atomic Transaction Record, coordinated with row-level Write Intent locks for global ACID consistency.

3. Hybrid Clock & Conflict Resolution: Clock drift is neutralized via HLC logical propagation combined with uncertainty interval waiting/restart protocols to guarantee snapshot isolation and linearizability.

4. Elastic Scaling & Online Upgrades: The decentralized Gossip protocol enables node discovery, while the Allocator balances Range replicas dynamically, enabling zero-downtime hardware failover and non-blocking online Schema changes.

L2 Data Flow Topology

• Client SQL Gateway Parse Lookup Route Range Locate Leaseholder

Raft Command Write Intent Create PENDING Transaction Record Raft

Consensus Commit Update Transaction Record to COMMITTED Async Intent

Cleanup Return OK.

Trade-off Matrix: Concurrency & Availability

Isolation vs Latency: Serializable Snapshot Isolation (SSI) $\rightarrow$ Weak Isolation (SI / RC)

SSI guarantees total protection against anomalies $\rightarrow$ but high-conflict write paths trigger frequent transaction aborts and Read Restarts [higher write latency, capped throughput]

Time Sync vs Availability: HLC (Physical + Logical) $\rightarrow$ Hardware TrueTime (GPS + Atomic Clocks) [HLC eliminates complex hardware dependencies and deployment limits $\rightarrow$ but requires Clock Offset uncertainty waits. Drift beyond limits causes immediate node panics [sacrifices local availability to enforce consistency]]

Transaction Protocol: 2PC Coordinator + Write Intents $\rightarrow$ 1PC Single-Range Commit Optimization [2PC enables arbitrary transactions spanning multiple Ranges $\rightarrow$ but 1PC accelerates single-Range commits by skipping the 2PC round-trip. Limit scope to drop multi-phase latency [narrows conflict window, improves small txn latency]]

Replication vs Heartbeats: Coalesced Heartbeats $\rightarrow$ Individual Raft Group Heartbeats [Coalescing slashes network traffic and CPU overhead from thousands of Raft heartbeats $\rightarrow$ but reduces the real-time refresh rate and health detection responsiveness of individual Ranges [lowers overhead, slightly slower failover detection]]

[M4.1] Hybrid Logical Clock (HLC) Physical and Logical Compound Formula

HLC bypasses GPS/atomic clock hardware. An HLC timestamp consists of physical time $L.i$ and logical counter $c.i$. If the local physical clock exceeds the highest known cluster clock, the physical value advances; if physical time is unchanged but causal events occur, the logical count $c.i$ increments, ensuring a strict causal partial order.

[M4.2] Clock Offset Uncertainty: Uncertainty Interval Read Restarts

Within the maximum clock offset drift limit (`max_offset`), if a read's 快照 timestamp T_{read} falls into the uncertainty interval $[T_{read}, T_{read} + \text{max_offset}]$ of another node, the transaction must execute a Read Restart, updating its timestamp to avoid read/write anomalies.

[M4.3] HLC State Machine: Tick and Update Protocols

- **Tick (Local event):** $l_i = \max(l_i, \text{physical_time})$, if physical time advances, reset $c_i = 0$; otherwise, increment $c_i = c_i + 1$.
- **Update (Message receive):** Upon receiving $T_{msg} = (l_{msg}, c_{msg})$ via RPC, update $l_i = \max(l_i, l_{msg}, \text{physical_time})$, resetting or merging logical counters accordingly to align cluster clocks.

[M4.4] Follower Reads: Local Stale Reads with HLC Alignment

Follower Reads allow read-only transactions to bypass the Leaseholder. Clients execute read requests at a historic timestamp $T_{read} \leq HLC_{local} - \text{max_offset}$. Since T_{read} is guaranteed to be older than any active uncertainty window, Followers can safely serve linearizable reads locally.

[M5.1] MVCC Physical Storage Layout and Multi-Version Key Encoding

The storage engine indexes multiple versions of row data. Modifications write new key-value pairs formatted as: `Key/Timestamp` $\rightarrow$ `Value`. Read transactions fetch the nearest committed version older than or equal to their 快照 timestamp, while obsolete historical versions are collected in the background.

[M5.2] Read Barrier and Snapshot Isolation (SI) Implementation

Read transactions carry an HLC timestamp T_{read} acting as a visibility barrier. During scans, if a transaction encounters a Write Intent with a timestamp $\leq T_{read}$, it blocks until the lock is resolved; if it encounters a version or intent $> T_{read}$, the read barrier ignores it to provide Snapshot Isolation.

[M5.3] Serializable Snapshot Isolation (SSI): Read Timestamp Cache

CockroachDB enforces Serializable isolation. The system uses a Read Timestamp Cache to track the maximum timestamp at which any key was read. If a write transaction attempts to update a key already read at a higher timestamp, it detects a read-write anti-dependency and aborts to prevent write skew.

[M5.4] MVCC Version Garbage Collection (GC TTL) and Storage Compaction

A background queue periodically scans Ranges to flag historical versions exceeding the Zone's Time-To-Live (GC TTL) for physical deletion. The storage engine (Pebble) permanently purges these keys during LSM-Tree compaction runs to reclaim disk space.

[M6.1] Gossip Protocol: Decentralized Peer-to-Peer Topology Discovery

Nodes join the cluster by connecting to seed addresses. Using a Gossip protocol, they asynchronously broadcast node health, locality tags (data centers, racks), storage utilization, and peer network latency metrics to build a global cluster topology map.

[M6.2] Fault-Domain Isolation: Rack-Aware Replica Placement Policies

The Allocator honors locality tags (fault domains) configured in the Zone settings. Replicas are distributed such that a Raft quorum (e.g., 2 out of 3 replicas) is spread across separate racks or datacenters, safeguarding cluster availability against rack or power grid outages.

[M6.3] Live Replica Rebalancing and Online Scaling

When Zone policies change, the Allocator evaluates migration paths. Replicas are added to target nodes using Raft membership changes, synchronizing WAL logs as Raft Learners. Once caught up, they promote to active Followers, and the obsolete replicas are decommissioned without disruption.

[M6.4] Online Schema Changes: Non-Blocking Table Upgrades

When adding columns or secondary indexes, CockroachDB transitions schema states through distinct phases: `Delete-only` (deletes index entries) $\rightarrow$ `Write-only` (updates index on writes) $\rightarrow$ `Public` (fully available). Historical data is backfilled in the background to avoid blocking table writes.

[M6.5] Leaseholder Failover: Stuck Node Mitigation and Lease Migration

If a Leaseholder experiences high disk IO queuing or packet loss, its Lease heartbeats will time out. Healthy Followers detect the missed lease renew interval and initiate lease acquisition to reroute client read/write traffic immediately, bypassing node restart delays.

[M6.6] Hot-Spot Mitigation: Hot Splits and Lease Distribution

If a Range's write/read QPS surpasses utilization limits, it is flagged as a hot-spot. The Allocator issues a split command (Hot Split) and schedules the newly spawned Leaseholder to a less utilized physical host, notifying clients to direct traffic to the new endpoints.

[M6.7] Clock Synchronization Monitoring and Auto-Panic Safety Fencing

Each node runs NTP synchronization checks. If a node detects that its local physical clock drift relative to the cluster quorum exceeds the `server.clock.max_offset` limit (200ms by default), it triggers an immediate Auto-Panic to prevent corrupt writes and guarantee data consistency.

Zone T1: CockroachDB Core Parameters

`kv.range.max_bytes = 64MB` : Maximum physical size of a Range before triggering a dynamic Split. `\server.clock.max_offset = 200ms` : Maximum allowed clock drift before a node triggers an Auto-Panic to fence itself. `\kv.transaction.max_intents_bytes = 256KB` : Memory limit for buffering Write Intents before flushing to disk to avoid OOM. `\sql.defaults.vectorization.enabled = true` : Enables CPU SIMD vectorization for DistSQL operators by default. `\kv.allocator.range_rebalance_threshold = 0.05` : Watermark capacity imbalance threshold (5%) to trigger Range migration.

Zone T2: Diagnostics & Troubleshooting